

# Modelo de detección de caídas con dispositivo basado en Arduino

P. de Miguel Pueyo<sup>1</sup>, M. Escudero Rodríguez<sup>1</sup>

<sup>1</sup> Escuela de Ingeniería de Fuenlabrada, Universidad Rey Juan Carlos, Madrid, España

\* P. de Miguel Pueyo, [p.demiguel.2020@alumnos.urjc.es](mailto:p.demiguel.2020@alumnos.urjc.es)

\* M. Escudero Rodríguez, [m.escudero.2020@alumnos.urjc.es](mailto:m.escudero.2020@alumnos.urjc.es)

*Abstract: Falls represent a serious public health concern, particularly for individuals over the age of 60. This study aimed to design a fall detection device using an Arduino microcontroller and an ADXL345 accelerometer. A database was created, and machine learning algorithms were applied to predict falls. The Gradient Boosting model achieved an accuracy of 89.78%. While areas for improvement were identified, this project highlights the potential of cost-effective fall prediction technology. It is suggested to optimize the Arduino program, explore alternative algorithms, and develop an application for faster communication with emergency services. These affordable solutions have the potential to enhance quality of life and save lives.*

## I. Antecedentes y motivación

Las caídas constituyen un problema de salud global con graves consecuencias tanto en términos de morbilidad como de mortalidad. Según la Organización Mundial de la Salud (OMS), se estima que anualmente se producen 37,3 millones de caídas que requieren atención médica, resultando en 684.000 muertes y una pérdida de 38 millones de años de vida ajustados por discapacidad (AVAD) [1]. Si bien este problema afecta a personas de todas las edades, los adultos mayores son particularmente vulnerables debido a su mayor fragilidad y disminución de las capacidades físicas.

Es importante destacar que el impacto de las caídas se acentúa en los países de ingresos medios y bajos, donde aproximadamente el 80% de los casos ocurren. Esto se debe a la falta de acceso a servicios de asistencia adecuados y a la incapacidad de costear soluciones convencionales, como los sistemas de teleasistencia o los botones de socorro. Además, las personas mayores, que constituyen el grupo más afectado por las caídas, suelen estar menos familiarizadas con la tecnología y más reacias a adoptarla, lo que aumenta su susceptibilidad a sufrir lesiones y limitaciones funcionales.

## II. Hipótesis y objetivos

Motivados por esta problemática, el objetivo principal de este estudio es proponer una solución sencilla y funcional utilizando tecnología de predicción de caídas. Se plantea la hipótesis de que es posible diseñar un dispositivo de detección de caídas de bajo costo, empleando recursos electrónicos accesibles y aprovechando el potencial de los algoritmos de aprendizaje automático.

En este trabajo, se describe la metodología empleada para lograr este objetivo, que consta de tres etapas fundamentales. En primer lugar, se busca medir una magnitud o señal que proporcione información relevante para la

predicción de caídas, siendo la aceleración la variable seleccionada en este estudio. Luego, se genera una base de datos utilizando un microcontrolador Arduino y un acelerómetro ADXL345 para capturar las señales de aceleración asociadas a diferentes acciones, como caídas, caminar y sentarse. Finalmente, se desarrolla un clasificador utilizando algoritmos de aprendizaje automático y se evalúa su desempeño en la detección de caídas.

Este trabajo contribuye al campo de la asistencia tecnológica al proponer una solución de bajo costo y accesible para la predicción de caídas. Los resultados obtenidos permiten identificar áreas de mejora y brindan perspectivas para futuras investigaciones en este ámbito. Se espera que esta tecnología pueda ser implementada en dispositivos portátiles y aplicaciones de fácil uso, mejorando la calidad de vida de las personas mayores y contribuyendo a la reducción de los riesgos asociados a las caídas.

## III. Metodología

### Punto 1: Selección de la magnitud de medición

En esta etapa de la metodología, se llevó a cabo la selección de la magnitud de medición más adecuada para la predicción de caídas. Se consideraron diversas opciones, como ultrasonidos, sensores ópticos y acelerómetros. Finalmente, se determinó que la aceleración sería la magnitud principal a medir debido a su capacidad para detectar cambios bruscos y rápidos en el movimiento corporal, características fundamentales en la detección de caídas y por los recursos con los que disponíamos.

Entre los diferentes tipos de acelerómetros disponibles en el mercado, se optó por utilizar el acelerómetro ADXL345 debido a su amplia disponibilidad, precisión, bajo costo y facilidad de integración con el sistema propuesto. Este acelerómetro posee tres ejes de medición (eje X, eje Y y eje Z) y proporciona una resolución de 13 bits, lo que permite una precisión adecuada para el propósito de este estudio.

## Punto 2: Configuración del sistema de medición

Para llevar a cabo la medición de la aceleración, se utilizó el microcontrolador Arduino Mega 2560 en conjunto con el acelerómetro ADXL345.

Se desarrolló un programa en el entorno de desarrollo de Arduino que permitía la comunicación con el acelerómetro ADXL345 y la lectura de los datos de aceleración en tiempo real. La comunicación con el sensor se realizó a través del protocolo de comunicación I2C, el cual permite la transmisión de datos de manera eficiente.

El programa en Arduino se encargó de configurar el acelerómetro para obtener lecturas de aceleración en los tres ejes de coordenadas y transmitirlas al ordenador a través del puerto serial. La principal herramienta de implementación fue la librería SparkFunADXL345 y las herramientas de comunicación serial que ofrece Arduino. Se escogió un flujo de Datos de 9600Bd para mantener compatibilidad con futuro software utilizado en el desarrollo. El programa de lectura es simple, manteniendo el acelerómetro constantemente enviando datos y el microcontrolador recibéndolos, de este modo el puerto Serial recibe continuamente una terna (X, Y, Z) de datos de aceleración. Este programa se mantenía guardado en la memoria ROM del microcontrolador Arduino, de este modo podía funcionar sin tener que estar conectado al software de Arduino. Estas lecturas fueron utilizadas posteriormente en las etapas de procesamiento y análisis de datos.

La elección del Arduino Mega 2560 y la configuración del sistema de medición permitieron obtener una plataforma de bajo costo y fácil implementación para la captura de datos de aceleración, sentando las bases para las etapas posteriores de la metodología.

## Punto 3: Transmisión de datos a Matlab

Una vez configurado el dispositivo y el programa desarrollado en Arduino, se procedió a transmitir dichos datos a Matlab para su posterior análisis y procesamiento. Se utilizó la biblioteca Serial de Matlab, que permite establecer una comunicación sencilla y directa con el Arduino a través del puerto serial.

El programa desarrollado en Arduino fue configurado para enviar los datos de aceleración a Matlab mediante el puerto serial. En Matlab, se implementó un script que recibía los datos de aceleración en tiempo real y los almacenaba en una estructura de datos adecuada para su posterior procesamiento. Se realizaron múltiples pruebas para asegurar la correcta transmisión de los datos y evitar pérdidas o distorsiones durante el proceso.

## Punto 4: Creación de la base de datos

Una vez que los datos de aceleración fueron recibidos y almacenados en Matlab, se procedió a la creación de la base de datos. Para ello, se llevaron a cabo simulaciones reales de

caídas y otras acciones, como marcha y sentarse, mientras se registraban las lecturas de aceleración durante intervalos de tiempo de aproximadamente 4 a 7 segundos.

Estas simulaciones se realizaron con el objetivo de obtener un conjunto de datos representativo y diverso que abarcara diferentes escenarios y tipos de movimiento. Se tomaron un total de 120 muestras de datos, distribuidas equitativamente entre los tres tipos de acciones (caída (Figura 1), marcha (Figura 2) y sentarse (Figura 3)), con un 33% de muestras para cada categoría.

En este estudio, se realizaron simulaciones de caídas utilizando dos sujetos diferentes. El sujeto 1 tenía un peso de 82 kg y una altura de 182 cm, mientras que el sujeto 2 tenía un peso de 65 kg y una altura de 178 cm. De las 120 muestras de simulación, la mitad fueron realizadas por el sujeto 1 y la otra mitad por el sujeto 2.

Cada muestra de datos consistió en tres columnas correspondientes a los ejes de coordenadas X, Y y Z, respectivamente, de las lecturas de aceleración. Además, se agregó una columna adicional que indicaba el tipo de acción realizada en cada muestra (fall, sit, walk).

Los datos recopilados de todas las simulaciones se almacenaron en un único archivo en formato CSV para facilitar su manipulación y procesamiento posterior. Este enfoque permitió contar con una base de datos estructurada que serviría como conjunto de entrenamiento para el desarrollo y evaluación de los modelos de predicción de caídas.

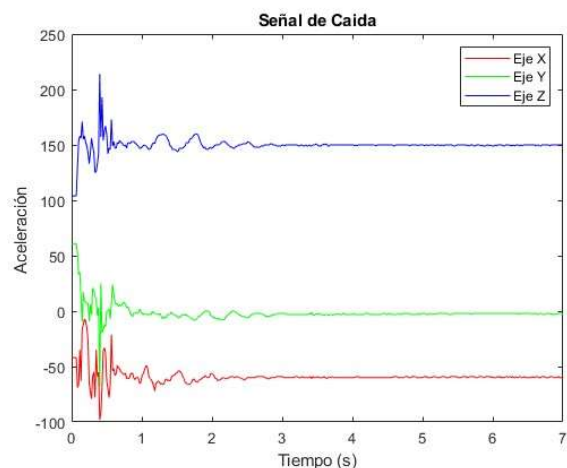


Figura 1: Ejemplo de simulación de caída

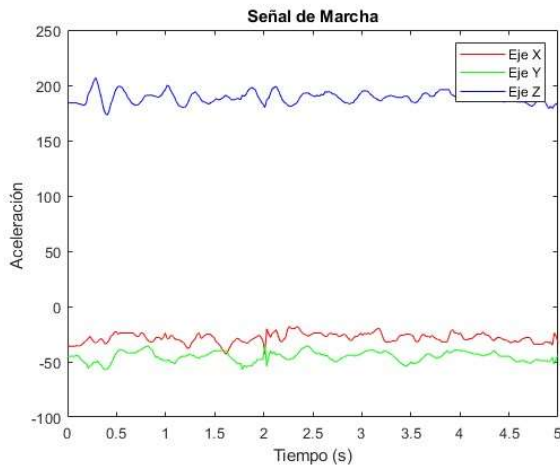


Figura 2: Ejemplo de simulación de marcha

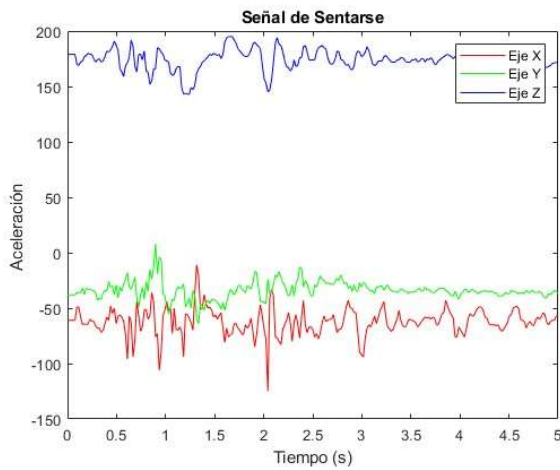


Figura 3: Ejemplo de simulación de sentarse

### Punto 5: Preprocesamiento de las señales

Una vez que se obtuvo la base de datos con las muestras de aceleración, se procedió al preprocesamiento de las señales con el objetivo de extraer características relevantes y preparar los datos para su posterior análisis y modelado.

A continuación, se calcularon una serie de características de las señales de aceleración para capturar información relevante sobre el movimiento. Estas características son la magnitud, la energía, la media, la varianza, el valor máximo y el valor mínimo de las señales.

Estas características fueron calculadas utilizando funciones y algoritmos disponibles en las bibliotecas NumPy y Pandas. El objetivo de este preprocesamiento fue reducir la dimensionalidad de los datos y extraer información significativa que pudiera ser utilizada por los algoritmos de aprendizaje automático para la predicción de caídas.

Después, se añadieron estas características a la base de datos total. Sin embargo, fueron las características de la señal obtenidas en esta etapa las que fueron utilizadas como entrada para los algoritmos de aprendizaje automático,

teniendo 6 variables o datos distintos para ajustar cada modelo de aprendizaje.

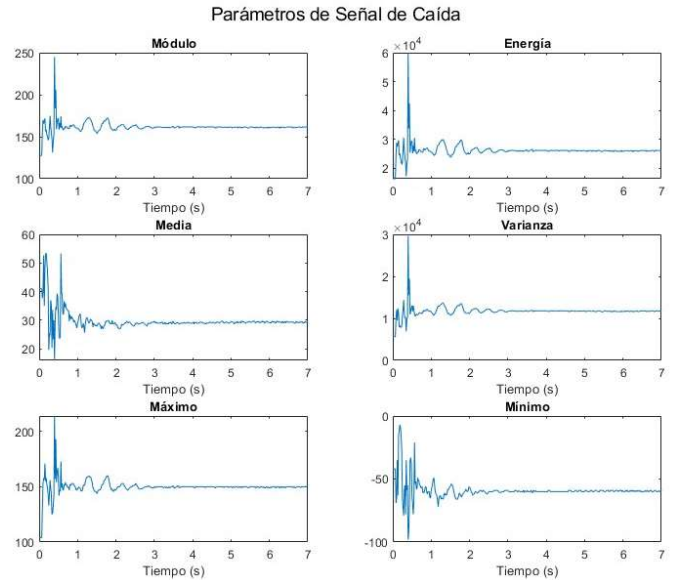


Figura 4: Ejemplo de las características de una simulación de sentarse

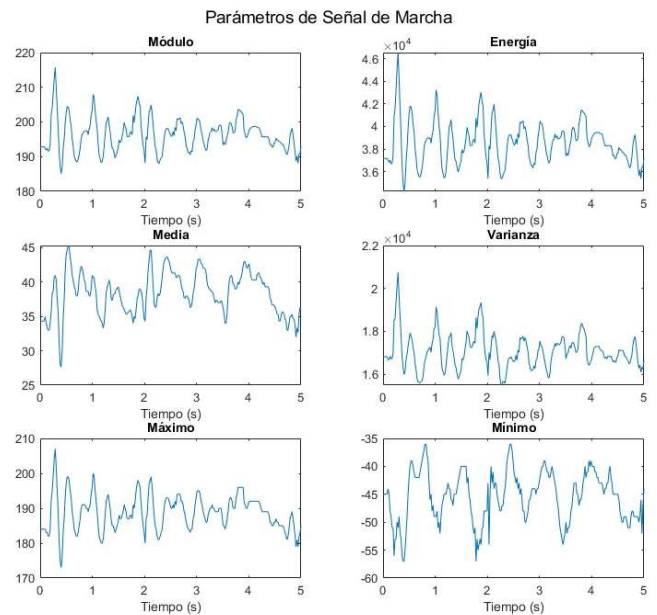


Figura 5: Ejemplo de las características de una simulación de marcha

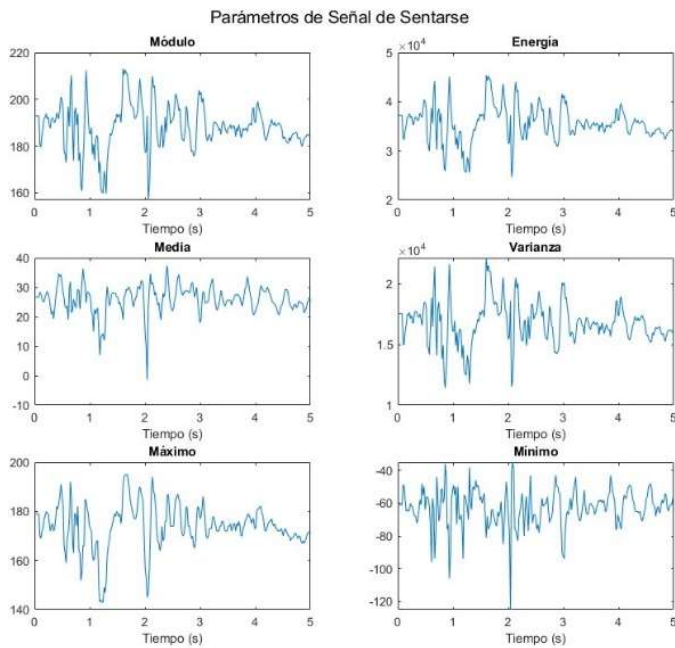


Figura 6: Ejemplo de las características de una simulación de sentarse

#### Punto 6: Estandarización y división de los datos

Una vez se obtuvieron las características de todas las señales mencionadas en el anterior punto, estas se estandarizaron mediante el método z-score (1), de este modo los datos de entrenamiento tienen la media 0 y desviación estándar 1, teniendo las mismas características y siendo óptimos para el entrenamiento de algoritmos [2].

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

Para el posterior entrenamiento y comprobación de modelos, se procedió a dividir el conjunto de datos obtenidos en dos partes: un 80% de los datos se destinó al entrenamiento del modelo y el 20% restante se reservó para evaluar su rendimiento como conjunto de prueba una vez realizado todo el ajuste de modelos y el entrenamiento de estos. Para garantizar una evaluación imparcial y evitar sesgos, se realizó una mezcla aleatoria de los datos antes de dividirlos.

Esta técnica asegura que tanto el conjunto de entrenamiento como el de prueba contengan una representación equilibrada de las distintas clases o tipos de datos, lo que permite una evaluación más confiable y generalizable del modelo de aprendizaje automático utilizado.

#### Punto 7: Evaluación y selección del modelo

En esta etapa, se llevó a cabo la evaluación y selección del modelo de predicción de caídas más efectivo. Se consideraron tres algoritmos de aprendizaje automático: Decision Tree, Random Forest y Gradient Boosting Classifier.

- **Decision Tree**: es un algoritmo de aprendizaje supervisado que divide el conjunto de datos en ramas y nodos de decisión basados en reglas condicionales. Cada nodo representa una

característica y cada rama representa una posible salida. El árbol se construye seleccionando la característica óptima que divide el conjunto de datos en las mejores ramas posibles. Una vez construido, el árbol puede ser utilizado para hacer predicciones siguiendo las reglas de decisión en los nodos.

- **Random Forest**: es un algoritmo de aprendizaje automático que combina múltiples árboles de decisión para realizar predicciones. Cada árbol en el bosque se construye utilizando una muestra aleatoria del conjunto de datos de entrenamiento y características seleccionadas de manera aleatoria. Luego, las predicciones de cada árbol se combinan mediante votación o promediado para producir una predicción final.

- **Gradient Boosting Classifier**: es un algoritmo de aprendizaje automático que combina múltiples modelos de aprendizaje débiles en un modelo fuerte mediante el enfoque de impulso o boosting. En cada iteración, el modelo se ajusta a los errores del modelo anterior, enfocándose en los casos más difíciles de clasificar. Esto se logra mediante el cálculo de los gradientes de pérdida y la actualización de los pesos de los casos de entrenamiento.

Para determinar el mejor modelo, se emplearon dos métodos de optimización de hiperparámetros: Grid Search CV (Cross Validation) y Random Search CV. Estos métodos permitieron explorar diferentes combinaciones de valores de hiperparámetros para cada algoritmo y determinar la configuración óptima que maximizara la precisión del modelo. En ambos casos se usaron 20 folds, esto implicó dividir el conjunto de datos de entrenamiento en 20 partes iguales, entrenar y evaluar el modelo en múltiples combinaciones de datos de entrenamiento y prueba.

- **Grid Search CV**: es una técnica utilizada para encontrar los mejores hiperparámetros de un modelo de aprendizaje automático. Consiste en definir una cuadrícula de posibles valores para los hiperparámetros y luego evaluar el rendimiento del modelo utilizando la validación cruzada para cada combinación de valores en la cuadrícula. El objetivo es encontrar la combinación de hiperparámetros que maximice la métrica de evaluación, como la precisión o el área bajo la curva.

- **Random Search CV**: es una técnica similar a Grid Search CV, pero en lugar de probar todas las combinaciones posibles, se realiza una búsqueda aleatoria de las combinaciones de hiperparámetros. En lugar de especificar una cuadrícula de valores, se define una distribución de probabilidad para cada hiperparámetro, y luego se muestrean valores aleatorios de estas distribuciones para formar una combinación de hiperparámetros. Esto permite explorar de manera más eficiente el espacio de hiperparámetros, especialmente cuando algunos hiperparámetros son más importantes que otros.



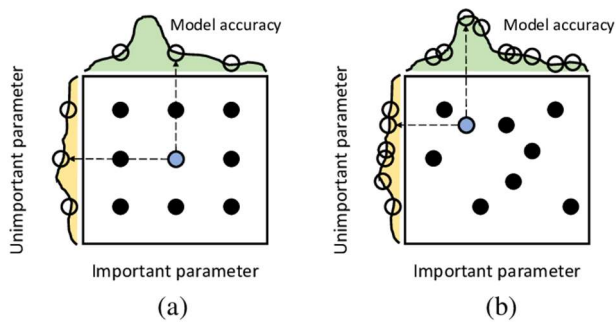


Figura 6: Comparación entre (a) Grid Search y (b) Random Search para el ajuste de hiperparámetros. Los nueve puntos indican los candidatos. Las curvas de la izquierda y de arriba indican la precisión del modelo. [3]

Todos los procesos de ajustes de hiperparámetros, normalización de datos, procesado etc... se realizó con la seed 42 mediante la librería Science Kit Learn.

Los hiperparámetros utilizados fueron:

- Max depth (MD): La profundidad máxima del árbol de decisión.
- Max features (MF): El número máximo de características a considerar al buscar la mejor división en un árbol.
- Min samples split (MSS): El número mínimo de muestras requeridas para dividir un nodo interno en un árbol de decisión.
- N estimators (NE): El número de árboles en un conjunto de bosques aleatorios o Gradient Boosting.
- Learning rate (LR): La tasa de aprendizaje utilizada en algoritmos de boosting para controlar la contribución de cada árbol.
- Criterion (Cr): La función utilizada para medir la calidad de una división en árboles de decisión, como "gini" o "entropía".

## IV. Resultados

Tras realizar la evaluación y emplear los métodos de optimización de hiperparámetros, los resultados obtenidos para cada uno de los algoritmos fueron:

### Decision Tree

|               | Cr          | MD          | MSS | Precisión |
|---------------|-------------|-------------|-----|-----------|
| Grid Search   | <i>gini</i> | <i>none</i> | 6   | 0.8677    |
| Random Search | <i>gini</i> | 7           | 2   | 0.8658    |

Tabla 1: Hiperparámetros óptimos para el algoritmo de Decision Tree según método de ajuste y resultados de precisión obtenidos

### Random Forest

|               | MD | MF          | MSS | NE  | Precisión |
|---------------|----|-------------|-----|-----|-----------|
| Grid Search   | 8  | <i>auto</i> | 4   | 500 | 0.8852    |
| Random Search | 8  | <i>sqrt</i> | 4   | 500 | 0.8852    |

Tabla 2: Hiperparámetros óptimos según método de ajuste para el algoritmo de Random Forest y resultados de precisión obtenidos

### Gradient Boosting

|               | LR  | MD | MSS | NE | Precisión |
|---------------|-----|----|-----|----|-----------|
| Grid Search   | 0.1 | 8  | 6   | 50 | 0.8952    |
| Random Search | 0.1 | 7  | 2   | 50 | 0.8934    |

Tabla 3: Hiperparámetros óptimos según método de ajuste para el algoritmo de Gradient Boosting y resultados de precisión obtenidos

Como se puede observar los resultados con mayor precisión fueron los establecidos por Grid Search CV. A continuación, con esos hiperparámetros se ajustaron los respectivos modelos y se llevó a cabo el test con el 20% de muestras que reservamos anteriormente. Finalmente, los resultados de las de precisión tras realizar el test fueron:

|                   |        |
|-------------------|--------|
| Decision Tree     | 0.8739 |
| Random Forest     | 0.8891 |
| Gradient Boosting | 0.8978 |

De esta manera, se encontró que el algoritmo Gradient Boosting Classifier obtuvo la mayor precisión, con un valor de 0.8978. Esto indica que este modelo fue el más efectivo para predecir las caídas utilizando las características extraídas de las señales de aceleración.

Sin embargo, en nuestra experiencia también fue el que tuvo un mayor coste de recursos y un mayor tiempo en el entrenamiento y respuesta, por lo que se deberá evaluar los pros y contras en costes computacionales, precisión y velocidad de respuesta de cada uno de los modelos.

## V. Discusión y conclusiones

En la sección de discusión, se identificaron varias posibles mejoras para el desarrollo y aplicación del sistema propuesto. En primer lugar, se sugiere optimizar el programa de Arduino y el almacenamiento de datos, explorando opciones para mejorar la comunicación con Matlab, como utilizar un sensor

más potente o aumentar la tasa de muestreo de los datos de cada simulación. La limitación de la comunicación a 9600 baudios podría ser superada para obtener una mayor precisión en las mediciones.

Además, se propone probar otros algoritmos de aprendizaje para determinar si existe alguno más adecuado para el propósito del estudio. Esto permitiría evaluar y comparar diferentes enfoques de aprendizaje automático en la detección de caídas.

Un aspecto a mejorar es la extracción de características útiles de los datos obtenidos, como datos de frecuencia u otros parámetros relevantes. Estas características adicionales podrían mejorar la precisión en la predicción de caídas.

El procesamiento de datos y la velocidad de realización de las predicciones se identificaron como áreas que requieren mejoras. Se sugiere implementar un procesamiento rápido de los datos, posiblemente mediante computación en la nube, para agilizar las predicciones de caídas y reducir el tiempo de reacción. Esta mejora permitiría utilizar el sensor en dispositivos wearables con conexión a Internet, brindando una mayor eficiencia y facilitando la comunicación rápida con los servicios de emergencia.

Una posibilidad adicional a considerar es utilizar PySpark en streaming para procesar los datos de caídas en tiempo real. PySpark es una biblioteca de procesamiento distribuido que permite el análisis eficiente de grandes volúmenes de datos. Al aplicar PySpark en este proyecto, se podría realizar la detección de caídas en tiempo real a medida que los datos llegan, lo que permitiría una respuesta inmediata ante emergencias. Esta aproximación aprovecha la capacidad de procesamiento paralelo y distribuido, mejorando la velocidad y la capacidad de respuesta del sistema. La implementación de PySpark en streaming facilitaría la detección ágil y precisa de caídas, y podría contribuir a desarrollar una aplicación o programa que detecte señales de caídas en tiempo real.

En cuanto al futuro del proyecto, se destaca el enorme potencial de los modelos de aprendizaje automático en el campo de la asistencia. Con las mejoras mencionadas, se vislumbra la posibilidad de desarrollar una aplicación de uso sencillo para personas mayores, junto con un dispositivo wearable de bajo coste, que permita una detección rápida de caídas y una comunicación eficiente con los servicios de emergencia. Esto tendría un impacto significativo en la salud y podría salvar vidas.

En resumen, se concluye que el estudio presenta perspectivas prometedoras en la detección de caídas, pero requiere mejoras en la optimización del programa, exploración de otros algoritmos de aprendizaje, extracción de características adicionales, y procesamiento más rápido de los datos. Con las mejoras propuestas, se espera que el proyecto pueda evolucionar hacia el desarrollo de una aplicación o programa capaz de detectar caídas basado en una señal de entrada en un tiempo de respuesta muy reducido, lo cual tendría un gran valor en el ámbito de la asistencia y la atención médica.

## DECLARACIÓN DEL AUTOR

Conflicto de intereses: Los autores declaran no tener ningún conflicto de intereses.

## REFERENCIAS

- [1] World Health Organization: WHO. (2021). Caídas. *www.who.int*. <https://www.who.int/es/news-room/fact-sheets/detail/falls>
- [2] sklearn.preprocessing.StandardScaler. (s. f.). scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [3] Pilario, K. E. S., Cao, Y., & Shafiee, M. (2021). A Kernel Design Approach to Improve Kernel Subspace Identification. *IEEE Transactions on Industrial Electronics*, 68(7), 6171-6180. <https://doi.org/10.1109/tie.2020.2996142>